Database Design Demonstration and Report

CSC7052

Contents

Overview

In alignment with Peter Chen's main principle that a database should reflect the real-life workings of a system (Chen, 1976), we undertook the design and implementation of a Relational Database Management System (RDBMS) for a commercial accommodation booking system, in this case hotels.com. This assignment mirrors selected aspects of the real accommodation booking system, reflecting how the procedures of the website may be recreated in our database whilst maintaining ACID principles and keeping Data Integrity.

The problem presented to us is to reverse engineer hotels.com into an RDBMS. This involved figuring out how to cover procedures such as hotel bookings and accounts, as well as representing availability.

Technologies

I used a range of different design, development, and data injection tools when designing the Entity Relationship Diagram (ERD) and building my database:

- MySQL - the chosen RDBMS for storing and relating data
- phpMyAdmin - the data administration tool for managing the database
- Visual Studio Code - an Integrated Development Environment (IDE) used as an SQL editor. I chose this too because it included various extensions and formatters, such as the ability to access my database from VSCode itself
- Chrome (Developer Tools) – this was used to reverse engineer the website: looking at API calls to see data structure, inspecting the HTML elements to view data presentation, and using the application tab to see libraries such as the Google Maps API
- Draw.io - used to create my ERD and represent relationships between tables, showing different forms of connections (i.e. one to many having different end styles) and highlighting Primary and Foreign keys

Design Assumptions

When reverse engineering the hotels.com website, I made some assumptions regarding the website's practices and procedures. These included:

- The hotel handles tax and fees. this assumption was made as when viewing prices, it states 'including taxes and fees'
- Search uses lowest price from the hotel's available rooms
- That capacity for each room type in a hotel being greater than or equal to one, given when booking with more than one room you can still use a single room type.
- Card details are encrypted and stored, therefore not using a third-party API. This assumption was made as you are given the option to save your card upon booking
- Children stays cost the same as adults
- Review score is a mean of all review scores; this is a common way of dealing with review averages.
- Reviews are stored in the database, not externally. I assumed this as developer tools on Chrome showed that reviews were managed internally
- Sale pricing is manually entered, not a base price multiplier
- The website uses a Map API; developer tools showed that hotels.com uses a Google Map API (Figure 1)

Out of Scope

As this project is limited in both time and scope, I was not able to implement everything that I would have liked to; therefore when first going through the website I decided that certain areas would be out of scope for the project. These included:

- Packages and flights
- Groups and Meetings
- Tickets and tourist attractions
- Children pricing
- Loyalty points
- Gift cards
- Coupon codes
- Bed preferences (chosen at booking)
- FAQs

- Surge pricing
- Landing Page

Discussion of group progression

As a group we first choose to focus on the act of entity discovery. We each took an entity from the website and created a table listing all the attributes associated with that entity. This was an important step as it allowed us to distinguish between what could be considered an entity and what could be considered an attribute. Once we had implemented the primary and foreign keys, these individual tables were brought together into an ERD diagram.

Our next step was the act of normalisation. We each took another look at the tables we had created and made key decisions on how to organise them. We ensured we had a database that made sense to use as a developer, but also prevented issues such as redundant or inconsistent data, or challenges faced when creating queries to access and update tables; the initial ERD was created using these principles.

Different issues regarding database design that we discussed as a group included:

- the problem of having the option of both an adult and a child and how this would affect the pricing
- pricing itself as a base concept: how and where to store the price, sale modifiers, room multipliers etc
- the way in which some hotels dynamically calculate pricing called surge pricing

A lot of the initial decisions were made together, however as I progressed through the project on my own and began designing the ERD and building the database, I found that a lot of the assumptions and design decisions between myself and the initial group discussions began to differ.

ERD

Design of the ERD followed the steps of normalisation which align with the values of Codd (E. F. CODD, 1970).

- 1NF – Tables had a primary key to provide uniqueness and made sure that the data was atomic (e.g., a hotel only had one address)
- 2NF – columns have no partial dependencies. Implemented by breaking tables out into a many-to-many, and linking a many-to-many to a many-to-one if required
- 3NF – use of lookup tables which broke out repeating data ensured 3NF was respected

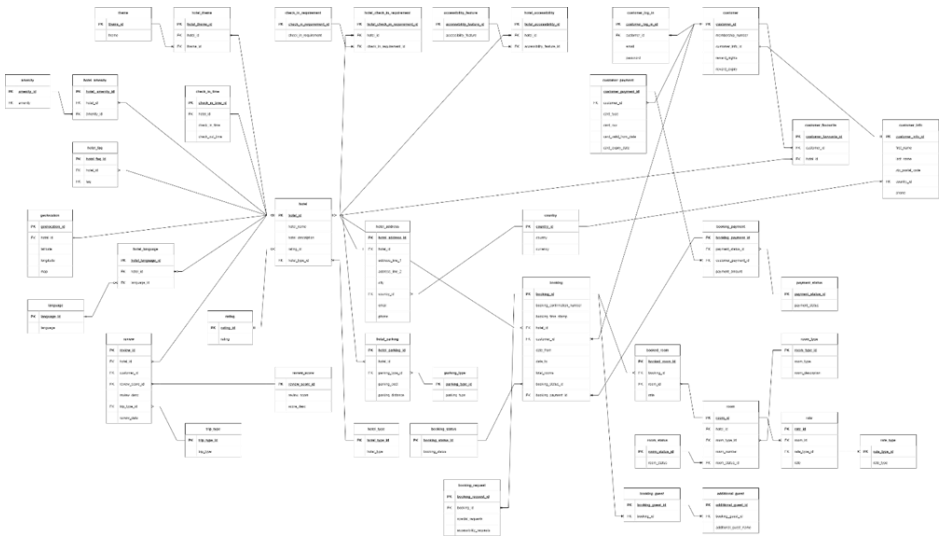


Figure 2: Initial Diagram

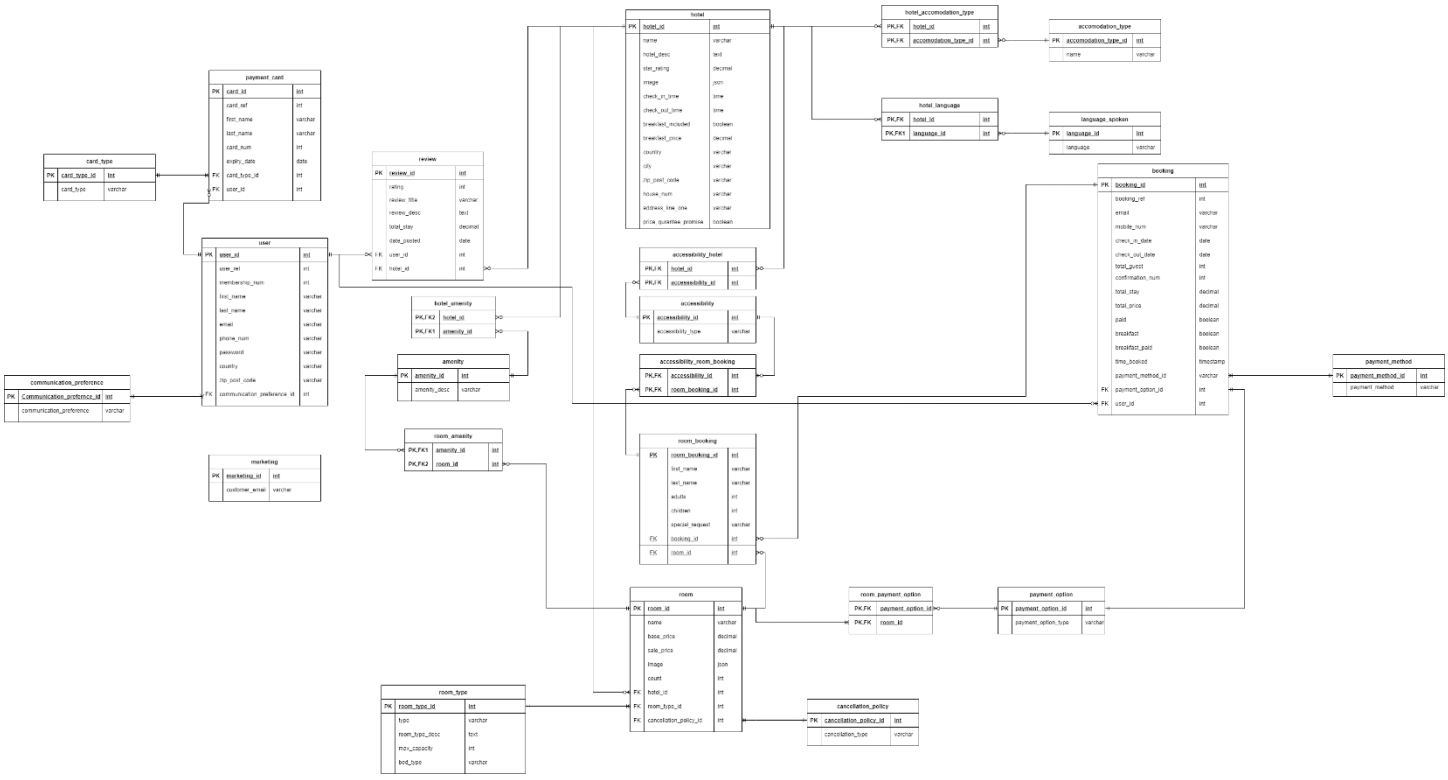


Figure 3: Final Design (Draw.io)

Individual ERD and Database Development

Design Decisions

Several important design decisions had to be made throughout this assignment:

- **Address table** - was not included in the database as hotels only have one address tied to them, and a user is only asked for their postcode and/or country
- **Nullable fields** - driven by optionality on the hotels.com site. Examples include user.country; user.zip_post_code; review.title; room_booking.special_request; hotel.breakfast_price; accessibility_room_booking.accessibility_id; accessibility_hotel.accessibility_id; booking.breakfast, booking.user_id and booking.breakfast_paid. Some hotels do not offer breakfast and therefore a price is not an option, nor is it an option to choose breakfast or state if it has been paid. Accessibility is not always provided by a hotel and therefore is not an option in the room_booking table or hotel_accessibility table. The field booking.user_id can be null as you are not required to have an account in order to make a booking
- **City/Country** – was not added as part of the database as city and country names can change and this would require internal upkeep. Additionally, a hotel can only be in one country, therefore there is no need for an additional table. This is a part of the online booking system that is better handled by an external API such as Google maps
- **Autogenerated fields** - membership_num and user_ref. These are seeded from characters, random ints, timestamps and are also concatenated, all of which occurs in a SQL query and is then inserted into the appropriate table in the database
- **Consistent naming conventions** - done using snake case in my table names and fields. I tried my best to be consistent with foreign key naming conventions, using the general syntax: FK_sourcename_targetname_field_id. I only used prefixes where MySQL reserved words became a problem, for example hotel_desc

Relationships

Many-to-Many relationships were enforced between tables through the use of bridging tables. Example tables include:

- hotel_accomodation_type – links together the hotel table and accommodation type table
- hotel_language – links together the hotel table and language_spoken table
- accessibility_hotel – links together the hotel table and the accessibility table
- accessibility_room_booking – links together the room_booking table and the accessibility table
- room_payment_option – links together the room table and the payment option table

One-to-Many relationships were enforced using lookup tables. Examples include:

- A room_type can have many rooms
- A cancellation_policy can be applied to many rooms
- A user can have many payment cards
- A communication preference can be applied to many users

By creating an amenity table, with a hotel_amenity and room_amenity table linked alongside it as a One-to-Many, we prevent duplication and produce consistent data.

Some relationships between tables are optional. Examples include:

- A user may only have one or many payment cards
- A hotel or room may have one or many amenities
- A hotel may speak one or many languages

Uniqueness and normalisation were maintained in the database through the existence of unique fields. Primary keys were autoincremented in the database to ensure uniqueness. There were some situations where two primary keys acted as a composite key within a bridging table. The creation of foreign key relationships ensured referential integrity. Specific fields were unique in some tables, for example a user's email.

Joins allow you to combine rows from multiple tables, as long as the connecting tables share a common field. Different types of joins allow you to alter how the result of the query is displayed in MySQL. I utilised INNER and LEFT joins throughout the assignment. There are four types of joins (Oracle, 2021):

- INNER – returns results that have matching values in both tables
- LEFT - returns all results from the left table, and matched records from the right table
- RIGHT - returns all results from the right table, and matched records from the left table
- OUTER - returns all results when there is a match in either the left or right table

Normalisation

A user can book multiple rooms in one booking; therefore I created a room_booking table, which acted as a bridging table for room and booking. Although you are asked for your accessibility requests at booking, accessibility requests are linked to rooms and hotels. An accessibility table with a list of choices, allowed many-to-many bridging tables to be created between the room_booking table and accessibility table and the hotel table and accessibility table.
A hotel and a room have their own selection of amenities; however, a room can inherit amenities that the hotel has. Therefore, to reflect this in the database I created a separate amenity table which then could link as a many-to-many to both the room and hotel table.
A marketing table stands alone within the database. This is populated upon account creation and contains a unique email for each user (Figure 4).

Availability

The database was designed in such a way that a hotel could have multiple of the same room type available, and that this could be reflected at the booking stage. To do so I created a room table, with a count field representing the number of said room type available at the hotel. I also created a bridging table named room_booking which contained a surrogate primary key of room_booking_id; this table allowed there to be multiple rooms per

booking. When a user did a search to see what rooms were available on the dates that they choose, check in and check out dates cannot overlap with the check in and check out dates of the bookings that were already present for each room; this was implemented in the search and hotel details queries.

Pricing

Pricing was calculated on the assumption that when a user does a search, the minimum price is displayed for each room which is available. The sale price was used in all queries; in order to assist queries the sale price defaulted to the base price if it was null. Pricing queries used aggregate functions: MIN was used to calculate the minimum price available for each hotel, MAX was used to check if a group had at least one of the properties. The booking price was based on total stay * rooms * sale cost. This was then stored as total_price in the booking table. (Figure 5)

SQL

Data Types

MySQL provides a variety of data types: these include Numeric Data Types; Date and Time Data Types; String Data Types; Spatial Data Types and the JSON Data Type.

- JSON - storing a list of image URLs in both the hotel and room table. This acts as a simple lookup in the frontend for gathering external images and then presenting
- Enum - can be a good choice of data type for simple non_reusable options, such as communication preference or payment method (Oracle, 2021). Whilst a lookup table is effective as one reusable list, it can lead to messy tabular design. I did not choose to use this data type, instead using only lookup tables
- Boolean - used several times in this project, which is converted to tinyint by the RDBMS and stored as a single bit. I used this for examples such as whether the booking had been paid or if the hotel offered breakfast or not. When the data is sent to the frontend the 1 and 0's would be interpreted as true or false
- Varbinary - used for passwords, in order to be able to encrypt and then store them as a hex value
- Text – used for descriptions, as you cannot be sure how long these may be. Examples included room_type_desc and hotel_desc, which used the Longtext type
- Decimal - used for fields such as price and star_rating, where I was able to make use of the formatting to set the number of significant figures and decimal places required
- Varchar - variable character length, utilised for fields such as first_name, last-name, house_num and phone_num. Also used in situations where the value could be both numbers and letters; certain values would be based on the hotel site field input restriction, e.g. the phone number text field on the site allows + characters, therefore is a varchar and not an int
- Date - utilised for fields such as check_in_date and check_out_date, where the date is required but not time

I only set specific lengths of data types where I knew for sure that it is how they are displayed. For example, a membership number has values of the Int data type of length 9.

Queries

- SELECT vs SET – SET is singular and requires only use of '=' as an assignment operator, whilst SELECT allows multiple sets using ':=' but returns all details with premade column names. This is used in the search query, where SELECT allows multiple variables to be listed, and SET is used in the context of the stay duration
- CONCAT_WS() - joins two or more strings together with a separator. I used this in the initial search query to display a list of hotel properties as address (using AS function). Using the CONCAT_WS allowed me to separate each individual field. I also used the standard CONCAT variant to generate a booking_ref (SET @booking_ref = CONCAT('BK', @confirmationnumber, UNIX_TIMESTAMP());)
- DATEDIFF() - takes the given dates and calculates the number of days between them. I used this in queries to calculate stay duration or total_stay (SET @stay_duration = DATEDIFF(@checkout, @checkin);). The parameters given were based on the check_in_date and check_out_date

- LAST_INSERT_ID() – this returns the last inserted id for the current user. This is used in the query which creates a booking, to get the room_booking_id from the last insert
- ISNULL() - returns the second value if the given value is null. If the given value is null, this is interpreted as true (Figure 6)
- IF() - checks the condition stated based on the parameters given and returns differing values of true or false. In the example it checked if the house_num was null, if this returned as true then the null value would be replaced with the hotel name (Figure 6)
- LIKE – checks for a string of a specified pattern within another string. Wild card characters (%) are used to allow any characters. This was used in a filter query which allowed a user to search for a specified pattern within a hotel name. (Figure 7)
- FORMAT() – restricts the number of decimal places. This was used to format the number of decimal places in the average review score to 1
- GROUP BY/HAVING – this allows aggregate comparison and filters down the results. This is used in the search query where it groups by hotel_id. (Figure 8)
- CONVERT() - converts a value into the specified datatype or character set. When AES_DECRYPT is used it returns a blob type which is unreadable, therefore the CONVERT() function is used to convert blob to text.
- Sub queries – a query nested within another query. Many of these were used in various queries throughout the database project.
- Aggregate functions - These types of functions can be utilised in a SELECT or HAVING statement, but not a WHERE statement. This is because the WHERE statement must execute before the GROUP BY.
  - MIN() – returns the smallest value of the group. I used it to calculate the minimum price that would be displayed based on your search criteria. (Figure 5)
  - MAX() – returns the largest value of the group, gets maximum count of rooms where free cancelation is true, and if it's greater than 0 it will be flagged as true and appear as free cancellation. (Figure 5)
  - AVG() – returns the mean value of a group. I used it to calculate the average review rating between the min and max parameters set. (Figure 10)
  - COUNT() - returns the total count of all found . I used this function to check if the number of current bookings for rooms in a given date range exceeds the count of rooms available. It calculates whether the count of rooms plus the additional rooms requested for is greater than the count of rooms. (Figure 8)

Filters

I wrote several queries for filter options, covering at least one of each type:

- Boolean - in which the user wants to know if the hotel includes breakfast (Figure 9)
- Text input - searching for a specific word in the hotel name (Figure 7)
- Ranged query - the user can input a range of guest ratings (Figure 10)
- Value checkboxes - the user may check what option they would like their hotel to have; I chose to demonstrate accommodation type and amenities (Figure 11)
- Numeric checkboxes - the user can search for hotels with a specific star rating (Figure 12)

Filter Design

When you execute a search query and are presented with a page of hotels, you are still given the option to filter the hotels using other parameters. When the website carries out a filter query, it is possible that one of two options occurs: either the main search query from before is resubmitted with modifiers referring to the parameters chosen, or the query uses hotel ids passed from the search pages original results. I choose to do the latter in my filter queries.

Search

When designing the search query, I built a single query to handle all the details required for each returned hotel row in the Search page. This could also be handled by multiple queries in serial, though for demonstration purposes I felt it would require a lot of assumptions on how the calling service would process the data. An alternative is using a simple query that sends back all the data required and the frontend filters down to the required details.

Transaction

When writing the query to create a booking, I wrapped the code with a Transaction statement. This is important as there are several queries happening at once; therefore this would fulfil the ACID properties, specifically that of Atomicity in which a transaction is either performed in its entirety or not at all. By adding a Commit statement at the end this fulfils the Durability property, in which once a transaction commits, its result is permanent. This is particularly important as it is essential as part of a hotel booking system that user bookings are safely stored and won't be lost. The use of a Transaction statement means a Savepoint can be added, allowing the developer to rollback this particular Savepoint if as failure occurs or the user decides to go in and cancel a booking.

Cryptography

When encrypting the user's account password I chose to use a SHA-2 rather than SHA-1, specifically SHA256. SHA-1 is a 160-bit encryption, whilst SHA-2 is a 256-bit checksum; this higher bit count means that SHA-2 is more secure than the latter (Oracle, 2021). In my query I used the rearrangement, salt, and hash method used in Scheme 2 of the hashing efficiency test (Sutriman, 2019), however I engineered a simpler version to demonstrate this. In the article 'Analysis of Password and Salt Combination Scheme To Improve Hash Algorithm Security', the author tests three schemes, each with a different salt and hash rearrangement. The results concluded that the preparation and addition of a few extra parameters can '…significantly increase the strength of the hash algorithm.' (Sutriman, 2019). Scheme 2 was the most effective as it showed the longest average attack time. This scheme rearranges the password and salt strings, appends the salt, and then hashes. I also made the decision to store the salt in the database, as if it was acquired by malicious individuals, it would not assist in brute forcing the passwords, as it is simply to prevent duplicate hash detection using known passwords.

PCI compliance security standards state that you must not '…store the card verification code or value (three-digit or four-digit number printed on the front or back of a payment card used to verify card-not present (transactions) after authorization.' (PCI Security Standards Council, LLC, 2018). This why I have made the decision not to store the security code (CVV) in the payment card table. When encrypting the card details of the user, I used the AES-Encrypt algorithm (Advanced Encryption Standard), a two-step algorithm. As is evident in the query associated with this step in the booking system, the card number is password locked. If you wish to decrypt this information, you must know the password used when the details were first encrypted; it is important to note that storing sensitive data such as a payment card is very bad practice. I am simply showing how this encryption process works for the purpose of the project, and due to the assumption that I have made at the beginning that when the user chooses to save their card at booking this information is then stored and encrypted in the database. It is much safer to outsource to someone external, such as Stripe or Chargebee.

A query in which encryption is effectively demonstrated can be seen in figure 13 and 14.

Improvements to Current Design

As this project is limited in both time and scope, I was not able to implement everything that I would have liked to. There are areas of the database that I did not have time to build or come up with reasonable solutions for.

The addition of review tiers would be a valuable improvement to the database. Each review rating appears to have a description attached to it:

- 10.0 = Exceptional
- 8.0 = Very good
- 6.0 = Good
- 4.0 = Fair

I would figure out how to implement this new information into the ERD and within the database. Once I concluded as to how this is to be done, I would be sure to update queries where applicable.

An addition of a sale modifier such as seasonal pricing would be a good improvement to the current design. I would create a pricing table with the list of seasonal periods and price modifier for each. I would then update my

queries using IF(), to check if the booking dates are within a seasonal period and then update the price query with the additional modifier. Surge pricing would be another possible improvement. This would be easy to implement as it would just be another modifier that if not required in a particular booking would default to 1. This means it would have no effect on the total_price if keep in the pricing queries. In a surge pricing system there is no surge until 50% of the room is booked in the hotel, after that each room being booked will add 5% of the last booked price for the hotel. In different seasons there will be an added percentage on top of the normal surge price (Raj Thakur, 2020).

A key improvement would be to implement the ability for a customer to book different types of rooms at once. This particular improvement came to mind when I came across the particular edge case seen in figure 15.
The price of the rooms is affected by the payment option, there is a sort of price tier. If a room has an option for non-refundable or free cancellation, choosing free cancellation will most likely increase the price of the room. As an improvement I would figure out how to implement this is the database.

Another account detail is that you are able to edit your password. I would improve the database by creating a query which allowed you to change your password, with considerations of salts and hashing and how these would be implemented in the query.

Overall I am content with the database I have created. Although there are many improvements that could be made, I think I have done a good job of reversing engineering the website and demonstrating how it would be applied to a real-life situation.

References

Chen, P., 1976. *The Entity-Relationship Model -Toward a Unified View of Data. ACM Transactions on Database Systems.* s.l.:s.n.

E. F. CODD, 1970. A Relational Model of Data for Large Shared Data Banks. 13(6).

Oracle, 2021. *11.3.5 The ENUM Type.* [Online] Available at: https://dev.mysql.com/doc/refman/8.0/en/enum.html

Oracle, 2021. *12.14 Encryption and Compression Functions.* [Online] Available at: https://dev.mysql.com/doc/refman/8.0/en/encryption-functions.html

Oracle, 2021. *13.2.10.2 JOIN Clause.* [Online] Available at: https://dev.mysql.com/doc/refman/8.0/en/join.html

PCI Security Standards Council, LLC, 2018. *Payment Card Industry (PCI) Data Security Standard, v3.2.1.* s.l.:s.n.

Raj Thakur, 2020. *Software System Design.* [Online] Available at: https://medium.com/software-system-design/price-surge-during-online-hotel-booking-ff060c33b2b6

Sutriman, 2019. Analysis of Password and Salt Combination Scheme. *(IJACSA) International Journal of Advanced Computer Science and Applications,* p. 422.

Appendix

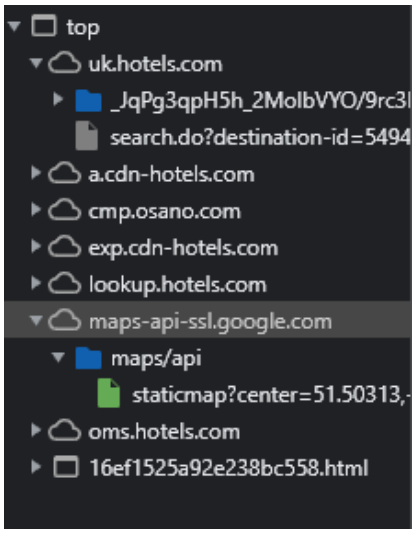


Figure 1: Evidence of Google Maps API

```
-- Add user to mailing list
SET
    @email = 'lnwacko@gmail.com';

INSERT INTO
    marketing (email)
VALUES
    (@email);

-- Get all users in mailing list
SELECT
    marketing.email
FROM
    marketing;

-- Remove user from mailing list by email
DELETE FROM
    marketing
WHERE
    marketing.email = @email;
```

Figure 4: populating the marketing table

```
 -- Get the lowest available price for hotel
    MIN(room.sale_price) * @stay_duration * @rooms AS total_price,
    @stay_duration AS stay_duration,
    MIN(room.sale_price) AS sale_price,
    room.base_price,
    -- Calculate savings percentage
    FORMAT(
        100 -(MIN(room.sale_price) / room.base_price * 100),
        2
    ) AS savings,
```

Figure 5: Pricing calculation

```
IF(
            ISNULL(hotel.house_num),
            hotel.name,
            hotel.house_num
        ),
        hotel.address_line_one,
        hotel.city,
        hotel.zip_post_code,
        hotel.country
    ) AS address,
```

Figure 6: Use of IF… ISNULL

```
-- Get name string from caller (search page)
SET
    @search = 'Hotel';

SELECT
    hotel.hotel_id
FROM
    hotel
WHERE
    hotel.name LIKE CONCAT('%', @search, '%')
```

Figure 7: Filter query for text input

```
-- Set search params from caller (search page)
SELECT
    @checkin := '2022-02-01',
    @checkout := '2022-02-11',
    @city := 'London',
    @guests := 2,
    @rooms := 4;

-- Calculate stay duration from checking dates
SET
    @stay_duration = DATEDIFF(@checkout, @checkin);
```

```sql
/* Complex query to gather all rows required for search page
 Could be done via multiple SQL calls, but this demonstrates
 completing the call entirely in one SQL command */
SELECT
    hotel.hotel_id,
    hotel.name,
    -- Build hotel address from fields
    CONCAT_WS(
        ', ',
        IF(
            ISNULL(hotel.house_num),
            hotel.name,
            hotel.house_num
        ),
        hotel.address_line_one,
        hotel.city,
        hotel.zip_post_code,
        hotel.country
    ) AS address,
    -- Get average review rating for hotel to one decimal place
    FORMAT(AVG(review.rating), 1) AS rating,
    -- Get the lowest available price for hotel
    MIN(room.sale_price) * @stay_duration * @rooms AS total_price,
    @stay_duration AS stay_duration,
    MIN(room.sale_price) AS sale_price,
    room.base_price,
    -- Calculate savings percentage
    FORMAT(
        100 -(MIN(room.sale_price) / room.base_price * 100),
        2
    ) AS savings,
    -- Check if payment at hotel is possible
    MAX(room_payment_option.payment_option_id = 2) > 0 AS payment_at_hotel,
    -- Check if free cancellation is available
    MAX(
        cancellation_policy.cancellation_type = "Free Cancellation"
    ) > 0 AS free_cancellation
FROM
    hotel
    INNER JOIN room ON room.hotel_id = hotel.hotel_id
    INNER JOIN room_type ON room.room_type_id = room_type.room_type_id
    LEFT JOIN cancellation_policy ON room.cancellation_policy_id =
cancellation_policy.cancellation_policy_id
    LEFT JOIN room_payment_option ON room.room_id = room_payment_option.room_id
    LEFT JOIN review ON review.hotel_id = hotel.hotel_id
WHERE
    hotel.city = @city
```

```
    AND room_type.max_capacity >= @guests
    AND (
        /* Check if the number of current bookings for
         rooms in given date range exceeds count of
         rooms available */
        room.room_id NOT IN (
            SELECT
                room_booking.room_id
            FROM
                booking
                INNER JOIN room_booking ON room_booking.booking_id = booking.booking_id
                INNER JOIN room ON room.room_id = room_booking.room_id
            WHERE
                -- Check if checkin dates are outside current bookings
                NOT (
                    (
                        booking.check_in_date < @checkin
                        AND booking.check_out_date < @checkin
                    )
                    OR (
                        booking.check_in_date > @checkout
                        AND booking.check_out_date > @checkout
                    )
                )
            GROUP BY
                room_booking.room_id
            HAVING
                COUNT(room_booking.room_id) + @rooms > room.count
        ) -- Check if room currently has no bookings
        OR hotel.hotel_id NOT IN (
            SELECT
                room.hotel_id
            FROM
                booking
                INNER JOIN room_booking ON room_booking.booking_id = booking.booking_id
                INNER JOIN room ON room_booking.room_id = room.room_id
        )
    )
GROUP BY
    hotel.hotel_id;

-- To sort, use ORDER BY <<query>> [ASC/DESC]
```

Figure 8: Search Query

```sql
-- Get all hotels that include breakfast
SELECT
    hotel_id,
    name
FROM
    hotel
WHERE
    breakfast_included = 1
    -- Hotels list passed from caller (search)
    AND hotel_id IN (1, 3, 19);
```

Figure 9: Filter query for breakfast included

```sql
-- Get min and max ratings from caller (search page)
SELECT
    @min := 3,
    @max := 8;

SELECT
    hotel.hotel_id
FROM
    review
    INNER JOIN hotel ON review.hotel_id = hotel.hotel_id
WHERE
    -- Hotels list passed from caller (search)
    hotel.hotel_id IN (1, 3, 19)
GROUP BY
    hotel.hotel_id
HAVING
    AVG(review.rating) BETWEEN @min
    AND @max
```

Figure 10: Filter query for guest rating

```sql
-- Get hotels that are a certain type of accomodation
SELECT
    hotel_id,
    name
FROM
    hotel_accomodation_type
WHERE
    accomodation_type_id IN (3,8)
    -- Hotels list passed from caller (search)
    AND hotel_id IN (1, 3, 19);
```

```sql
-- Get hotels that have given amenities
SELECT
    hotel_id
```

```
FROM
    hotel_amenity
WHERE
    amenity_id IN (9,10)
    -- Hotels list passed from caller (search)
    AND hotel_id IN (1,3,19);
```

Figure 11: Filter query for accomodation type and amenities

```
-- Get hotels that have star rating in set
SELECT
    hotel_id,
    name
FROM
    hotel
WHERE
    star_rating IN (4,5)
    -- Hotels list passed from caller (search)
    AND hotel_id IN (1, 3, 19);
```

Figure 12: Filter query for star rating

```
/* DO NOT DO THIS, USE AN EXTERNAL PROVIDER SUCH AS STRIPE
 OR CHARGEBEE */
/* Get the cardholder details to be stored */
SELECT
    @user_id := 33,
    @first_name := 'Lauren',
    @last_name := 'McCann',
    @expiry_date := '2024-11-01',
    @card_type := 2,
    @pass := 'hot_enc2811' AS definitely_not_a_password,
    @card_num := '7915467815497645',
    @card_encrypt := AES_ENCRYPT(@card_num, @pass) AS card_encrypted;

/* Insert the record with the encrypted data */
INSERT INTO
    payment_card (
        user_id,
        first_name,
        last_name,
        card_num,
        expiry_date,
        card_type_id
    )
VALUES
    (
        @user_id,
```

```
        @last_name,
        @last_name,
        @card_encrypt,
        @expiry_date,
        @card_type
    );

-- Get card number back from account
SELECT
    card_id,
    first_name,
    last_name,
    expiry_date,
    card_type_id CONVERT(AES_DECRYPT(@card_num, @pass) USING utf8) AS card_num
FROM
    payment_card
WHERE
    card_id = 1;
```

Figure 13: Add payment card (Encryption)

```
/* Creates a base user account, either called
 from the create account page or during a booking */
SELECT
    @firstname := 'Lauren',
    @lastname := 'McCann',
    @email := 'lmwacko@gmail.com',
    @communicationpreference := 2,
    /* BAD PRACTICE, PASSWORDS SHOULD BE HASHED EXTERNALLY
     AND THEN SENT TO PREVENT SENDING PLAIN TEXT PASSWORDS OVER
     THE INTERNET */
    @plain_password := 'password' as definitely_not_a_password,
    /* Rearrangement and salting technique taken from scheme 2 of
     https://www.academia.edu/60398997/Analysis_of_Password_and_Salt_Combination_Scheme_To_
Improve_Hash_Algorithm_Security
     Create a random salt in SHA256, six chars long */
    @salt := SUBSTRING(SHA2(RAND(), 256), 1, 6) as salt;

-- Simplified rearrangement in specified pattern (placing salt inside)
SET
    @rearranged = CONCAT(
        SUBSTRING(@plain_password, 1, 10),
        @salt,
        SUBSTRING(@plain_password, 10)
    );

-- Append salt to rearrangement
SET
```

```
    @concat_rearrange = CONCAT(@rearranged, @salt);

-- Hash the salt and rearrangement combination with SHA256
SET
    @stored_salted_hash = SHA2(@rearranged, 256);

-- Store this user in the database
INSERT INTO
    user (
        first_name,
        last_name,
        email,
        communication_preference_id,
        user_password,
        salt
    )
VALUES
    (
        @firstname,
        @lastname,
        @email,
        @communicationpreference,
        @stored_salted_hash,
        @salt
    );

SET
    @user_id = LAST_INSERT_ID();

SET
    @user_ref = CONCAT('U', @user_id, UNIX_TIMESTAMP());

SET
    @membershipnum = FLOOR(RAND() *(999999999 -1 + 1)) + 1;

UPDATE
    user
SET
    user_ref = @user_ref,
    membership_num = @membershipnum
WHERE
    user_id = @user_id;
```

Figure 14: Create a user account (Encryption)

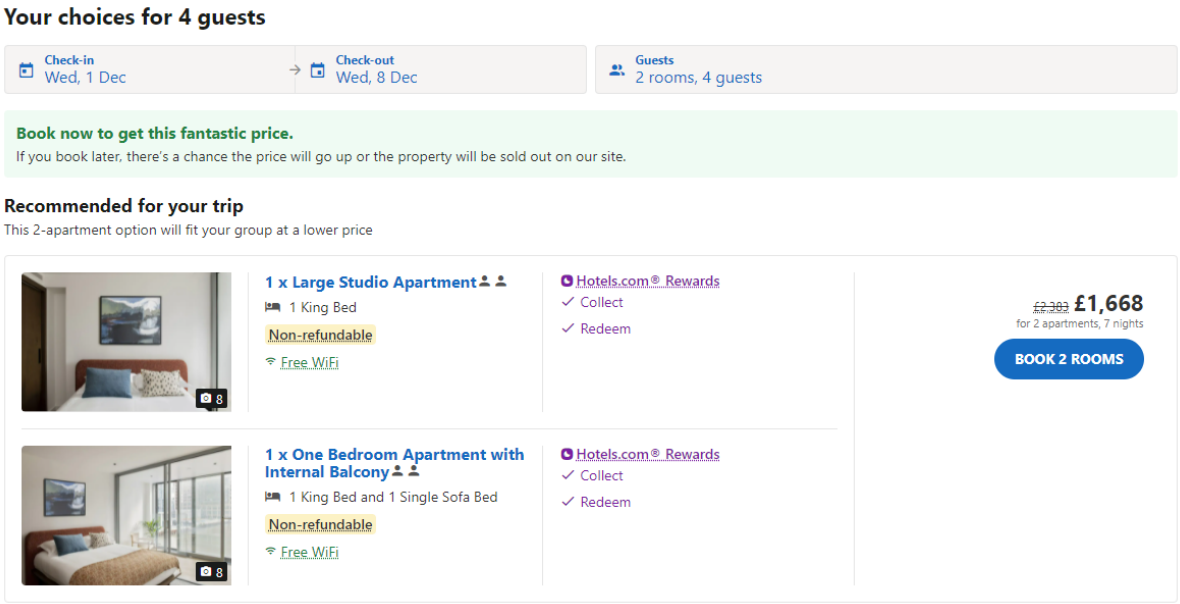
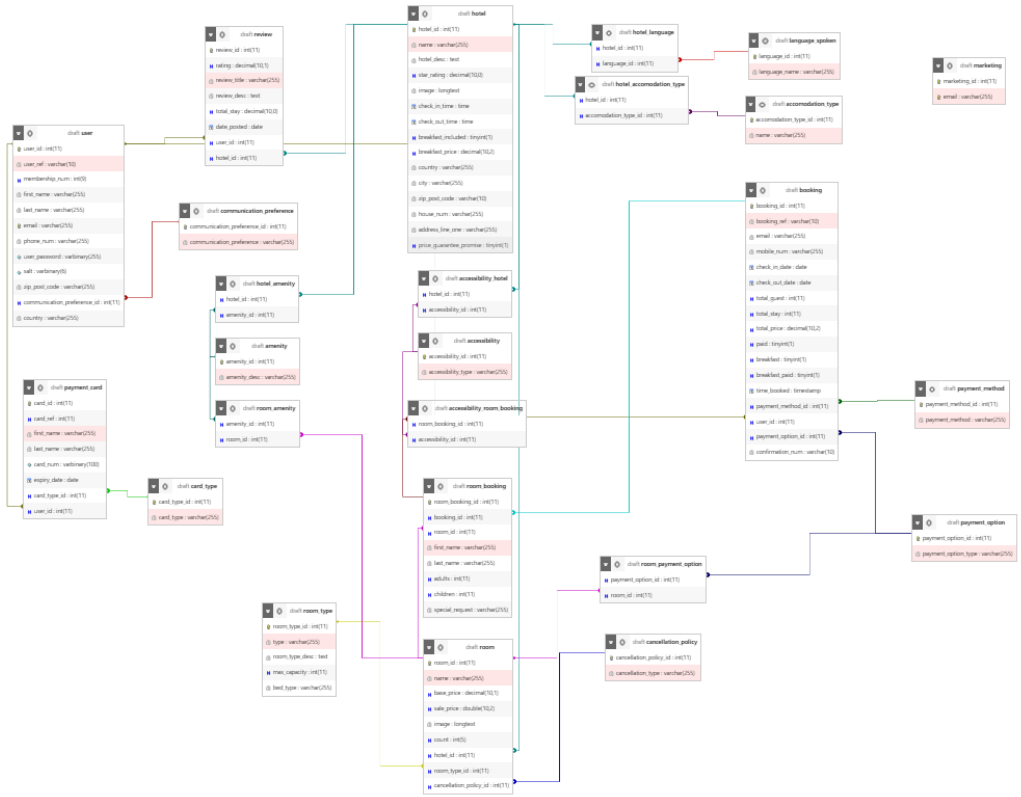


Figure 15: Edge case



Final Diagram (Designer View)